

Plan de Implementacion Duty of Care

- PLAN DE IMPLEMENTACIÓN: DUTY OF CARE EN PNPTV
 - Del Enunciado al Código — Práctico, Aplicable y Medible
 - Resumen ejecutivo
 - Estado actual — Lo que ya está (no hay que construir)
 - FASE 1 — Soberanía de privacidad (geobloqueo por creador)
 - FASE 2 — Transparencia de ganancias verificable por el creador
 - FASE 3 — Identidad verificable sin custodiar PII (migración hacia Verifiable Credentials)
 - FASE 4 — Offboarding ético (protocolo de salida)
 - FASE 5 — Soberanía comunitaria: panel de transparencia público
 - Cronograma sugerido
 - Métricas de éxito global del plan
 - Lo que NO está en este plan (y por qué)
 - Apéndice: cómo testear cada fase antes de ir a producción

PLAN DE IMPLEMENTACIÓN: DUTY OF CARE EN PNPTV

Del Enunciado al Código — Práctico, Aplicable y Medible

Versión: 1.0

Fecha: Agosto 2026

Basado en: Auditoría directa del repositorio + tesis académica revisada

Audiencia: Equipo técnico PNptv (Santino, Carlos, Miguel)

Resumen ejecutivo

La auditoría técnica reveló que PNptv ya implementó los pilares de bienestar más complejos del Duty of Care (Wellness Mode, Use Tracker, § 2257, filtro contextualizado). Las brechas que quedan son las de **soberanía del trabajador**: el creador no puede ocultar su perfil de su propio país, no puede salir sin depender de que PNptv exporte sus credenciales, y no puede verificar el ledger de ganancias sin pedirle un reporte a un admin.

Este plan cierra esas brechas. Está organizado en 3 fases de impacto creciente, cada una con entregables medibles en código y métricas observables en producción.

Estado actual — Lo que ya está (no hay que construir)

Área	Módulo	Estado
Verificación de identidad § 2257	<code>identityVerificationService.js</code>	LIVE
Verificación de edad por IA	<code>ageVerificationService.js</code>	LIVE
Passkeys / WebAuthn (sin contraseña)	<code>routes.js</code> + <code>webAppController.js</code>	LIVE
Wellness Mode con cooling-off 24h	<code>wellnessModeService.js</code>	LIVE
Use Tracker (slam/smoke)	<code>routes.js</code> tabla <code>use_tracker_logs</code>	LIVE
Self-Care Center (sin ads, sin algo)	<code>SelfCareCenter.tsx</code>	LIVE
Filtro de contenido PNP-contextualizado	<code>contentModerationFilter.js</code>	LIVE
Sistema de reporte (8 categorías + CSAM auto)	<code>userReportService.js</code>	LIVE
Ban multi-vector	<code>platformBanService.js</code>	LIVE
Cashout multi-crypto (BTC/DASH/USDT/Meru)	<code>cashoutService.js</code>	LIVE
Ledger Ru\$h append-only	<code>tokenLedgerService.js</code>	LIVE

Área	Módulo	Estado
Derecho al olvido self-service	<code>selfEraseAccount</code> en <code>usersController</code>	LIVE
Fuzzing de ubicación HMAC	<code>nearbyService.js</code>	LIVE
Sistema de apelación público	<code>appealService.js</code>	LIVE

FASE 1 — Soberanía de privacidad (geobloqueo por creador)

Prioridad: CRÍTICA

Esfuerzo estimado: 3-5 días

Responsable: _____

Due date: _____

Impacto: Protege a creadores de ostracismo familiar, persecución en países hostiles, pérdida de empleo alternativo

Por qué ahora

El país de origen es el vector de riesgo más común para creadores en Latinoamérica. Un creador en Colombia, México o Venezuela que tiene familia o empleo convencional no puede controlar si un familiar que busca su nombre en PNptv llega a su perfil. Esta funcionalidad es la más solicitada en plataformas similares y la más ausente.

1.1 — Tabla de configuración de geobloqueo

```
-- Migration: add creator geo-privacy settings
ALTER TABLE users ADD COLUMN IF NOT EXISTS geo_block_countries TEXT[] DEFAULT '{}';
```

```
-- Example: ['CO', 'MX', 'VE'] – ISO 3166-1 alpha-2 codes
-- Empty array = visible globally (default, no change in behavior)

CREATE INDEX IF NOT EXISTS idx_users_geo_block ON users USING
    GIN(geo_block_countries)
WHERE array_length(geo_block_countries, 1) > 0;
```

1.2 — Middleware de geobloqueo en perfiles públicos

Archivo a modificar: `apps/backend/bot/api/middleware/ipTracker.js` (o crear `geoBlockMiddleware.js` nuevo)

```
// apps/backend/services/geoBlockService.js
'use strict';

const { query } = require('../config/postgres');
const logger = require('../utils/logger');

/**
 * Resolves country code from request using existing geo detection.
 * Falls back to null if unresolvable.
 */
function resolveCountry(req) {
    // geo detection already exists in subscriptionController – centralize here
    const forwarded = req.headers['x-forwarded-for'];
    const ip = forwarded ? forwarded.split(',')[0].trim() : req.ip;
    // Use maxmind/ipinfo if configured, else fall back to req.geo set by
    // upstream middleware
    return req.geo?.country || req.session?.user?.country || null;
}

/**
 * Check if a creator's profile is blocked for a given country.
 * Returns true if the requesting country is in the creator's block list.
 */
async function isBlockedForCountry(creatorId, requestingCountry) {
    if (!requestingCountry) return false; // Can't determine country – allow

    const { rows } = await query(
        `SELECT geo_block_countries FROM users WHERE id = $1 LIMIT 1`,
        [creatorId]
    );

    const blockList = rows[0]?.geo_block_countries || [];
    return blockList.includes(requestingCountry.toUpperCase());
}
```

```

/**
 * Express middleware factory for geo-blocking a creator's public profile.
 * Usage: router.get('/profile/:creatorId', geoBlockMiddleware('creatorId'),
 *           handler)
 */
function geoBlockMiddleware(creatorIdParam = 'creatorId') {
  return async (req, res, next) => {
    const creatorId = req.params[creatorIdParam] || req.query[creatorIdParam];
    if (!creatorId) return next();

    const country = resolveCountry(req);

    try {
      const blocked = await isBlockedForCountry(creatorId, country);
      if (blocked) {
        // Return the same 404 as a non-existent profile – never confirm the
        // profile exists
        return res.status(404).json({ error: 'Profile not found' });
      }
      next();
    } catch (err) {
      logger.error('geoBlockMiddleware error (fail-open)', { creatorId,
        error: err.message });
      next(); // Fail open – don't lock users out on a DB error
    }
  };
}

module.exports = { resolveCountry, isBlockedForCountry, geoBlockMiddleware };

```

1.3 — API para que el creador configure su lista

Archivo a modificar: `apps/backend/bot/api/routes.js` — agregar en la sección de rutas de creadores

```

// En routes.js, en la sección de creator settings
app.get('/api/webapp/creator/geo-privacy', requireSessionAuth,
  asyncHandler(async (req, res) => {
    const { rows } = await pool.query(
      'SELECT geo_block_countries FROM users WHERE id = $1',
      [req.session.user.id]
    );
    return res.json({ blockedCountries: rows[0]?.geo_block_countries || [] });
  }));

```

```

app.put('/api/webapp/creator/geo-privacy', requireSessionAuth,
  asyncHandler(async (req, res) => {
    const { countries } = req.body;
    if (!Array.isArray(countries)) return res.status(400).json({ error:
      'countries must be an array of ISO codes' });

    // Validate: ISO 3166-1 alpha-2 only, max 50 countries
    const VALID_ISO = /^[A-Z]{2}$/;
    const validated = countries
      .map(c => String(c).toUpperCase().trim())
      .filter(c => VALID_ISO.test(c))
      .slice(0, 50);

    await pool.query(
      'UPDATE users SET geo_block_countries = $1 WHERE id = $2',
      [validated, req.session.user.id]
    );
    return res.json({ success: true, blockedCountries: validated });
  }));

```

1.4 — UI en Creator Settings

Archivo a modificar:

`apps/web/src/pages/creators/CreatorSettings.tsx`

Agregar sección “Privacidad geográfica” con: - Selector de países con banderas (lista ISO completa, multiselect) - Toggle rápido “Ocultar perfil en mi país de origen” (auto-detectado por IP al moment de configurar) - Advertencia: “Tu perfil mostrará ‘no encontrado’ para visitantes de los países seleccionados”

1.5 — Aplicar el middleware en rutas de perfil público

Archivos a modificar: Routes donde se expone el perfil público de un creador (buscar `GET /api/webapp/profile/:username` o similar).

Métrica de éxito

- **Técnica:** 0 errores 5xx al activar geo-block; perfil retorna 404 desde IP del país bloqueado

- **Adopción:** $\geq 15\%$ de creadores activos configura al menos 1 país bloqueado en los primeros 30 días
- **Privacidad:** 0 reportes de “mi familia encontró mi perfil” post-implementación (vs. baseline actual)

FASE 2 — Transparencia de ganancias verificable por el creador

Prioridad: ALTA

Esfuerzo estimado: 2-3 días

Responsable: _____

Due date: _____

Impacto: El creador puede auditar su propio split sin pedirle nada a un admin

Por qué ahora

El ledger Ru\$h (`token_ledger`) existe y es append-only. El problema: el creador no tiene una vista de esto en la UI. Actualmente

`CreatorEarnings.tsx` muestra totales pero no el ledger transacción por transacción. Esto crea dependencia de confianza (“confío en que el 70% es el 70%”) en lugar de verificación directa.

2.1 — Endpoint de ledger propio del creador

Archivo a modificar: `apps/backend/bot/api/routes.js`

```
// Creator earnings ledger – full transaction history, paginated
app.get('/api/webapp/creator/earnings/ledger', requireSessionAuth,
  asyncHandler(async (req, res) => {
    const userId = req.session.user.id;
    const page = Math.max(1, parseInt(req.query.page) || 1);
    const limit = Math.min(100, parseInt(req.query.limit) || 50);
    const offset = (page - 1) * limit;
```

```

const { rows } = await pool.query(
  `SELECT
    tl.id,
    tl.created_at,
    tl.reason,
    tl.balance_delta,
    tl.gifted_delta,
    tl.balance_after,
    tl.source_type,
    tl.source_id,
    tl.metadata
  FROM token_ledger tl
  WHERE tl.user_id = $1
  ORDER BY tl.created_at DESC
  LIMIT $2 OFFSET $3`,
  [userId, limit, offset]
);

const { rows: countRows } = await pool.query(
  'SELECT COUNT(*) AS total FROM token_ledger WHERE user_id = $1',
  [userId]
);

return res.json({
  ledger: rows,
  total: parseInt(countRows[0].total),
  page,
  limit,
  commissionRate: 0.30,
  creatorRate: 0.70,
});
});

```

2.2 — Vista de ledger en CreatorEarnings

Archivo a modificar:

`apps/web/src/pages/creators/CreatorEarnings.tsx`

Agregar tab “Historial completo” con tabla paginada: - Columnas: Fecha, Concepto, Tokens recibidos, Balance después - Exportar a CSV (botón) - Filtros: por tipo (tip, contenido, suscripción, cashout) - Tooltip explicativo del split 70/30 en cada fila de `membership_purchase` / `content_purchase`

2.3 — Resumen de ganancias por categoría

```
// Summary endpoint – quick stats for the earnings dashboard
app.get('/api/webapp/creator/earnings/summary', requireSessionAuth,
  asyncHandler(async (req, res) => {
    const userId = req.session.user.id;

    const { rows } = await pool.query(
      `SELECT
        reason,
        COUNT(*) AS count,
        SUM(balance_delta) AS total_tokens,
        MIN(created_at) AS first_at,
        MAX(created_at) AS last_at
      FROM token_ledger
      WHERE user_id = $1 AND balance_delta > 0
      GROUP BY reason
      ORDER BY total_tokens DESC`,
      [userId]
    );

    return res.json({ breakdown: rows, tokenRateUsd: 1/6 });
  }));
```

Métrica de éxito

- **Técnica:** Endpoint responde < 200ms para ledgers con hasta 10,000 filas
 - **Adopción:** $\geq 40\%$ de creadores activos visita el ledger al menos 1 vez/mes
 - **Confianza:** Reducción de tickets de soporte tipo “no me pagaron bien” en $\geq 50\%$ (medible vs. baseline de Slack #ext-*)
-

FASE 3 — Identidad verificable sin custodiar PII (migración hacia Verifiable Credentials)

Prioridad: MEDIA-ALTA

Esfuerzo estimado: 2-4 semanas (incluye integración externa)

Responsable: _____

Due date: _____

Impacto: Elimina el mayor punto de concentración de PII sensible de la plataforma

Por qué ahora

Los documentos de identidad almacenados en el servidor de PNPTv (`id_document_path` , `id_selfie_path`) son el mayor riesgo de privacidad de la plataforma. Una brecha en este punto expone documentos gubernamentales de performers adultos — el peor escenario posible de doxxing institucionalizado.

3.1 — Migración completa a Persona (eliminar upload directo)

Persona.com ya está integrado en `identityVerificationService.js` (`startPersonaInquiry` , `handlePersonaWebhook`). El flujo alternativo (upload manual de documentos) debe ser deprecado.

Pasos: 1. Deshabilitar las rutas de upload directo de documentos en producción 2. Redirigir 100% del flujo de verificación a `startPersonaInquiry` 3. Los campos `id_document_path` e `id_selfie_path` en `creator_2257_records` quedan como referencias al `persona_inquiry_id` (ya implementado), no como rutas de archivos locales 4. Purgar los archivos existentes en disco con un script de migración auditado

Script de migración:

```

// scripts/migrate-2257-to-persona-refs.js
// Reemplaza rutas de archivo locales con referencias a Persona inquiry
// Solo para registros ya approved – los documentos son innecesarios una vez
// aprobado
const { query } = require('../config/postgres');

async function migrate() {
  const { rows } = await query(
    `SELECT id, user_id, id_document_path, id_selfie_path, persona_inquiry_id
    FROM creator_2257_records
    WHERE verification_status = 'approved'
    AND persona_inquiry_id IS NOT NULL`
  );

  for (const record of rows) {
    // Null out local paths – Persona is the authoritative record
    await query(
      `UPDATE creator_2257_records
      SET id_document_path = NULL, id_selfie_path = NULL
      WHERE id = $1`,
      [record.id]
    );
    // Log for audit trail
    console.log(`Migrated record ${record.id} for user ${record.user_id}`);
  }
  console.log(`Migrated ${rows.length} records.`);
}
migrate();

```

Métricas de éxito: - 0 nuevos archivos de documentos en disco post-migración - Persona integration rate $\geq$ 95% de nuevas verificaciones - Tiempo de verificación median $\leq$ 48h (vs. actual que depende de revisión manual)

3.2 — Exportación de datos portátil para el creador (data portability)

El creador debe poder exportar todos sus datos en un formato portable antes de cerrar su cuenta.

Archivo a modificar: `apps/backend/bot/api/routes.js`

```

// Data export – GDPR-style, creator-initiated
app.post('/api/webapp/account/export', requireSessionAuth, asyncHandler(async
  (req, res) => {
    const userId = req.session.user.id;

    // Rate limit: 1 export per 24h
    const lastExport = await redis.get(`data_export:${userId}`);
    if (lastExport) return res.status(429).json({ error: 'Export available once
      per 24h' });

    // Gather all creator data
    const [profile, ledger, content, earnings, subs] = await Promise.all([
      pool.query('SELECT username, first name, last name, email, bio,
        created_at FROM users WHERE id = $1', [userId]),
      pool.query('SELECT * FROM token_ledger WHERE user_id = $1 ORDER BY
        created_at', [userId]),
      pool.query('SELECT id, title, media_type, created_at, is_premium FROM
        creator_media WHERE creator_id = $1', [userId]),
      pool.query('SELECT * FROM creator_earnings WHERE creator_id = $1 ORDER BY
        created_at', [userId]),
      pool.query('SELECT * FROM creator_subscriptions WHERE creator_id = $1',
        [userId]),
    ]);

    const exportData = {
      exported_at: new Date().toISOString(),
      profile: profile.rows[0],
      ledger: ledger.rows,
      content: content.rows,
      earnings: earnings.rows,
      subscriptions: subs.rows,
    };

    await redis.setex(`data_export:${userId}`, 86400, '1');

    res.setHeader('Content-Disposition', `attachment; filename="pnptv-data-${
      userId}.json"`);
    res.setHeader('Content-Type', 'application/json');
    return res.json(exportData);
  }));

```

UI: Botón “Exportar mis datos” en `CreatorSettings.tsx` y en la pantalla de borrado de cuenta.

Métricas: - Funcionalidad disponible sin dependencia de soporte -
 Tiempo de generación < 5 segundos para el 95% de los creadores

FASE 4 — Offboarding ético (protocolo de salida)

Prioridad: MEDIA

Esfuerzo estimado: 1-2 semanas

Responsable: _____

Due date: _____

Impacto: Cubre la única brecha de RH clásico que el código no aborda: la salida

Por qué importa

El código actual tiene `selfEraseAccount` (borrado duro, irreversible). Pero no tiene un protocolo de salida que incluya: entrega de ganancias pendientes, aviso a suscriptores activos, período de gracia para descargar contenido propio. Un creador que borra su cuenta hoy pierde ganancias en hold si las hay.

4.1 — Pre-flight de borrado de cuenta

Archivo a modificar: `apps/backend/bot/api/controllers/usersController.js` — modificar `selfEraseAccount`

```
// Agregar pre-flight check antes de borrar
app.get('/api/webapp/account/erase-preflight', requireSessionAuth,
  asyncHandler(async (req, res) => {
    const userId = req.session.user.id;

    const [earnings, subs, hold] = await Promise.all([
      pool.query(
        'SELECT COALESCE(SUM(amount_creator),0) AS available FROM
         creator_earnings WHERE creator_id=$1 AND status=\'available\'',
        [userId]
      ),
      pool.query(
        'SELECT COUNT(*) AS active FROM creator_subscriptions WHERE
         creator_id=$1 AND expires_at > NOW()',
        [userId]
      ),
      pool.query(
```

```

        'SELECT COALESCE(SUM(amount_creator),0) AS held FROM creator_earnings
        WHERE creator_id=$1 AND status=\'held\'',
        [userId]
    ),
    []);

    return res.json({
      availableEarnings: parseFloat(earnings.rows[0].available),
      heldEarnings: parseFloat(hold.rows[0].held),
      activeSubscribers: parseInt(subs.rows[0].active),
      warning: parseInt(subs.rows[0].active) > 0
        ? `Tienes $
          {subs.rows[0].active} suscriptores activos. Serán notificados y
          reembolsados automáticamente.`
        : null,
      earningsWarning: parseFloat(earnings.rows[0].available) > 0
        ? `Tienes $$${earnings.rows[0].available} disponibles para retiro. Haz
          cashout antes de borrar tu cuenta.`
        : null,
    });
  });
});

```

4.2 — UI de offboarding en pasos

Archivo a modificar: Agregar flujo en

`apps/web/src/pages/Settings.tsx` o crear `OffboardingFlow.tsx`

Paso 1: Pre-flight (mostrar ganancias pendientes, suscriptores activos)

Paso 2: Opciones de cashout one-click si hay balance

Paso 3: Confirmación con campo de texto “DELETE MY ACCOUNT”

Paso 4: Confirmación bilingual (ES/EN)

4.3 — Notificación a suscriptores al borrar cuenta

Archivo a modificar: `apps/backend/bot/api/controllers/usersController.js` — en `hardDeleteUser`

Agregar notificación a suscriptores activos antes del borrado:

```

// En hardDeleteUser, antes de DELETE FROM users:
const { rows: activeSubs } = await client.query(
  'SELECT subscriber_id FROM creator_subscriptions WHERE creator_id=$1 AND
  expires_at > NOW()',
  [userId]
)

```

```
);  
  
// Notify each subscriber  
for (const sub of activeSubs) {  
  await sendSystemDM('system', sub.subscriber_id,  
    'Un creador que seguías ha cerrado su cuenta en PNptv. Tu suscripción ha  
    sido cancelada y el tiempo no utilizado será acreditado.',  
    client  
  );  
}
```

Métricas de éxito: - 0 tickets de soporte de creadores que borraron cuenta sin retirar ganancias - 100% de suscriptores notificados cuando un creador se va - Tiempo de offboarding (desde inicio del flujo hasta borrado confirmado): mediana < 10 minutos

FASE 5 — Soberanía comunitaria: panel de transparencia público

Prioridad: BAJA-MEDIA

Esfuerzo estimado: 1 semana

Responsable: _____

Due date: _____

Impacto: Verificabilidad externa del modelo; relevante para la tesis académica y para reguladores

Por qué importa

El documento académico señaló que la promesa de modelo cooperativo requiere transparencia verificable. No hace falta un ledger on-chain para dar este paso; basta con publicar estadísticas de distribución que cualquier observador pueda leer.

5.1 — Endpoint de transparencia pública

```
// Public transparency endpoint — no auth required
app.get('/api/public/transparency', asyncHandler(async (req, res) => {
  // Cache 1 hour
  const cached = await redis.get('transparency:stats');
  if (cached) return res.json(JSON.parse(cached));

  const [split, wellness, moderation] = await Promise.all([
    pool.query(`
      SELECT
        COUNT(DISTINCT creator_id) AS active_creators,
        SUM(amount_creator) AS total_paid_to_creators,
        SUM(amount_platform) AS total_platform_commission,
        ROUND(AVG(amount_creator / NULLIF(amount_creator + amount_platform,
          0)) * 100, 1) AS avg_creator_pct
      FROM creator_earnings
      WHERE status = 'paid_out'
        AND created_at > NOW() - INTERVAL '90 days'
    `),
    pool.query(`
      SELECT
        COUNT(*) AS wellness_sessions,
        SUM(wellness_days_accumulated) AS total_wellness_days
      FROM users
      WHERE wellness_days_accumulated > 0
    `),
    pool.query(`
      SELECT category, COUNT(*) AS reports
      FROM user_reports
      WHERE created_at > NOW() - INTERVAL '90 days'
      GROUP BY category
      ORDER BY reports DESC
    `),
  ]);

  const data = {
    updated_at: new Date().toISOString(),
    period: 'last_90_days',
    creator_economy: {
      active_creators: parseInt(split.rows[0].active_creators),
      creator_share_pct: parseFloat(split.rows[0].avg_creator_pct) || 70,
      platform_commission_pct: 30,
      total_paid_to_creators_usd:
        parseFloat(split.rows[0].total_paid_to_creators) || 0,
    },
    wellness: {
      total_wellness_sessions: parseInt(wellness.rows[0].wellness_sessions),
    },
  };
});
```

```

        total_wellness_days_accumulated:
            parseInt(wellness.rows[0].total_wellness_days),
    },
    moderation: {
        reports_by_category: moderation.rows,
    },
    },
    });

    await redis.setex('transparency:stats', 3600, JSON.stringify(data));
    return res.json(data);
}));

```

5.2 — Página pública de transparencia

Archivo: `apps/web/src/pages/Transparency.tsx` (nueva — excepción al no-new-files: es una página de contenido público, no un componente)

Mostrar las métricas del endpoint en una página accesible sin login, referenciable desde la tesis y desde comunicados de prensa.

Métricas de éxito: - Página indexada por Google - Citada en al menos 1 documento académico o regulatorio - Datos actualizados cada hora sin intervención manual

Cronograma sugerido

Semana	Fase	Entregable
1	Fase 1	Migración SQL + middleware geoblock + API endpoint
2	Fase 1	UI Creator Settings + QA en staging
3	Fase 2	Ledger endpoint + UI CreatorEarnings
4	Fase 3	Migración Persona 100% + script purga documentos
5-6	Fase 3	Data portability export + QA

Semana	Fase	Entregable
7	Fase 4	Pre-flight offboarding + UI offboarding flow
8	Fase 5	Transparency endpoint + página pública

Métricas de éxito global del plan

Métrica	Cómo medirla	Target 90 días post-lanzamiento
Creadores con geobloqueo activo	<code>SELECT COUNT(*) FROM users WHERE array_length(geo_block_countries,1) > 0</code>	≥ 15% de creadores activos
Creadores que visitaron su ledger	Analytics en CreatorEarnings (nueva tab)	≥ 40% MAU creadores
Tickets de soporte por “no me pagaron bien”	Conteo en Slack #ext-*	Reducción ≥ 50%
Documentos locales en disco post-migración	<code>find /opt/pnptvapp/uploads -name "*.jpg" -newer migration-date</code>	0 nuevos documentos
Offboardings con cashout previo	<code>SELECT COUNT(*) FROM cashout_requests WHERE created_at BETWEEN erase_started AND erase_completed</code>	≥ 80% de creadores con balance > \$0
	HTTP 200 en <code>/transparency</code>	Sí, sin SLA breach

Métrica	Cómo medirla	Target 90 días post-lanzamiento
Página de transparencia activa		

Lo que NO está en este plan (y por qué)

Ítem	Razón de exclusión
SSI/DIDs/ZKP nativos	Tecnología en maduración; complejidad de integración 3-6 meses; Persona ya cubre el 90% del beneficio práctico
Botón de pánico stealth	PNPtv es plataforma digital, no coordina trabajo presencial; riesgo físico in-person no es el caso de uso primario
Ledger on-chain (blockchain)	Costo de gas + latencia + UX pésima para el usuario promedio; el ledger DB append-only con exportación CSV es suficiente para verificabilidad práctica
Estructura cooperativa formal	Requiere reforma societaria, no código; fuera del alcance del equipo técnico
ARL colombiana / seguridad social	Requiere licencia de intermediación laboral; fuera del alcance técnico

Apéndice: cómo testear cada fase antes de ir a producción

Fase 1 — Geobloqueo

```
# Simular request desde Colombia (CO)
curl -H "X-Forwarded-For: 181.51.0.1" \
      https://pnptv.app/api/webapp/profile/[username_con_CO_bloqueado]
# Esperado: 404

# Simular request desde otro país
curl -H "X-Forwarded-For: 50.1.2.3" \
      https://pnptv.app/api/webapp/profile/[username_con_CO_bloqueado]
# Esperado: 200 con perfil
```

Fase 2 — Ledger

```
# Verificar que el total del ledger cuadra con creator_earnings
docker exec pg-pnptv psql -U pnptvbot -d pnptvbot -c "
SELECT
  tl.total_ledger,
  ce.total_earnings,
  ABS(tl.total_ledger - ce.total_earnings) AS discrepancy
FROM
  (SELECT SUM(balance_delta) AS total_ledger FROM token_ledger WHERE
   user_id='[id]' AND reason LIKE '%receive%') tl,
  (SELECT SUM(amount_creator) AS total_earnings FROM creator_earnings WHERE
   creator_id='[id]') ce;
"
```

Fase 3 — Documentos

```
# Verificar que no quedan rutas locales en registros aprobados
docker exec pg-pnptv psql -U pnptvbot -d pnptvbot -c "
SELECT COUNT(*) FROM creator_2257_records
WHERE verification_status='approved'
AND (id_document_path IS NOT NULL OR id_selfie_path IS NOT NULL);
```

"

Esperado: 0

Este plan fue generado con base en auditoría directa del repositorio de producción en agosto 2026. Cada bloque de código es aplicable directamente al stack actual (Node.js + Express + PostgreSQL + Redis + React/Vite). No se propone ningún cambio de tecnología ni arquitectura.